

Neural Networks and the Transformer: Attention from First Principles

Unit 2 of the Generative AI and LLM crash course

GenAI crash course

2026-10-09

Contents

Session map	2
1. Terms: the term tree	3
2. Remember	4
3. Understand: the derivation	5
3.1 Tier 1 (easy): intuition and a picture	5
3.2 Tier 2 (medium): the definitions made precise	6
3.2.1 The recurrent network and its limits	6
3.2.2 Attention, computed by hand	7
3.2.3 Self-attention, cross-attention, masks	7
3.3 Tier 3 (medium-hard): the main derivations	7
3.3.1 Why divide by $\sqrt{d_k}$	7
3.3.2 Multi-head attention costs no more than one big head	7
3.3.3 Positional encoding, and why the sinusoid gives relative offsets	8
3.3.4 Residuals, layer norm, and the parameter count of a block	8
3.4 Tier 4 (hard): cost, and the three Transformer families	10
3.4.1 Where the FLOPs go	10
3.4.2 Encoder, decoder, encoder-decoder	10
3.5 Tier 5 (hardest): open questions	12
4. Apply	12
4.1 Worked problems	12
4.2 Code and figures	12

5. Analyze	13
5.1 Limits	13
5.2 Dominance map	13
5.3 Sensitivity	13
5.4 Invariants	13
5.5 Failure modes	13
6. Evaluate	14
7. Create	14
8. Visualization lesson	14
9. Problem set	15
10. Retrieval practice and recap	15
11. Where next	16

Session map

Outcome of this session

After this session you can:

1. explain why recurrent networks struggle with long sequences, with a measured number, not a slogan;
2. compute scaled dot-product attention by hand for a small example, and say what every symbol means;
3. derive why attention divides by $\sqrt{d_k}$, why positional encoding is needed, and why the sinusoidal one gives *relative* offsets;
4. count the parameters and FLOPs of a Transformer block from its shape, and find the sequence length where attention starts to dominate the cost;
5. say which of BERT, GPT and T5 is an encoder, a decoder, or both, and why that decides what each is good at.

- **Prerequisites:** matrices and vectors, the dot product, softmax (defined in Section 2 below), a first neural-network course. Unit 3, which follows, uses the Transformer as a black box that outputs one score per vocabulary word; this unit opens the box.
- **Difficulty ladder used:** easy $\rightarrow$ medium $\rightarrow$ medium-hard $\rightarrow$ hard (Sections 3.1 to 3.4); the hardest tier is posed as open questions in Section 3.5.
- **Deliverables:** this document (.tex, .pdf, .docx), 6 figures, the module `codes/llm/transformer_np` / (attention, multi-head, positional encoding, a full block, RNN gradients, FLOP counts; 9 tests), and the *Attention heatmap* and *Positional encoding* widgets in the LLM Studio playground.

- **Source anchors:** Vaswani et al. 2017, arXiv:1706.03762 (Sections 3.1 to 3.5 and Table 1 for architecture, scaling by $\sqrt{d_k}$, sinusoidal encoding, complexity; Table 3 for the base-model sizes); Devlin et al. 2018, arXiv:1810.04805 (BERT); Raffel et al. 2019, arXiv:1910.10683 (T5); Su et al. 2021, arXiv:2104.09864 (rotary position embedding). All IDs were checked against arXiv on the day of writing, and the numbers quoted from Vaswani et al. were read from the paper’s PDF.

1. Terms: the term tree

Term	Symbol	One-line meaning	Where it appears	New / known
softmax		turns a list of scores into probabilities that sum to 1	2	new
sequence model		a model whose input and output are ordered lists	3.1	new
recurrent neural network (RNN)	h_t	reads a sequence step by step, carrying a hidden state	3.2	new
hidden state	h_t	the RNN’s running summary of everything read so far	3.2	new
vanishing gradient		gradients shrink exponentially with distance	3.2	new
attention		a weighted average of values, with weights from query-key similarity	3.2	new
query, key, value	Q, K, V	what I look for / what I offer / what I hand over	3.2	new
self-attention		queries, keys and values all come from the same sequence	3.2	new
cross-attention		queries from one sequence, keys and values from another	3.2	new
multi-head attention	h	several attention operations in parallel, then mixed	3.3	new
positional encoding	PE	vectors added to embeddings so the model can tell order	3.3	new
causal mask		forbids attending to later positions	3.2	new
residual connection		output = input + sub-layer(input)	3.3	new
layer normalisation		rescales each token vector to zero mean and unit variance	3.3	new
feed-forward network (FFN)	d_{ff}	the per-token two-layer MLP inside each block	3.3	new
encoder / decoder		bidirectional reader / left-to-right writer	3.4	new

Terms *used but not taught here*: SentencePiece and tokenization details (Unit 3), the KV cache (Unit 10), instruction tuning (Unit 6).

2. Remember

Softmax

$\text{softmax}(x)_i = e^{x_i} / \sum_j e^{x_j}$ turns any list of real scores x into positive numbers that add up to 1, giving the largest score the largest share. Worked example: $x = (2, 1, 0)$ gives $(e^2, e^1, e^0) / (e^2 + e^1 + e^0) = (7.389, 2.718, 1) / 11.107 = (0.665, 0.245, 0.090)$. Attention applies it to each row of scores.

Attention

Given queries $Q \in \mathbb{R}^{n_q \times d_k}$, keys $K \in \mathbb{R}^{n_k \times d_k}$ and values $V \in \mathbb{R}^{n_k \times d_v}$,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \quad (1)$$

Row i of the softmax is a probability distribution over the n_k positions: how much query i reads from each. The output is that weighted average of the value rows.

Multi-head attention

Split the width d into h heads of size $d_k = d/h$. Each head has its own projection matrices $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times d_k}$. The heads' outputs are concatenated and mixed by $W^O \in \mathbb{R}^{d \times d}$:

$$\text{MHA}(x) = [\text{head}_1; \dots; \text{head}_h]W^O, \quad \text{head}_i = \text{Attention}(xW_i^Q, xW_i^K, xW_i^V).$$

Sinusoidal positional encoding

For position pos and pair index $i = 0, \dots, d/2 - 1$ (Vaswani et al., Section 3.5):

$$PE_{pos,2i} = \sin(pos/10000^{2i/d}), \quad PE_{pos,2i+1} = \cos(pos/10000^{2i/d}). \quad (2)$$

A Transformer block (pre-norm form)

$$\begin{aligned} x' &= x + \text{MHA}(\text{LN}(x)), \\ y &= x' + \text{FFN}(\text{LN}(x')), \quad \text{FFN}(u) = \phi(uW_1 + b_1)W_2 + b_2, \end{aligned} \quad (3)$$

with $W_1 \in \mathbb{R}^{d \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d}$, usually $d_{ff} = 4d$. (The original paper puts the layer norm *after* the residual sum; the pre-norm form above is what most current models use.)

Memory hooks.

- *Library.* The **query** is your question, each **key** is the label on a shelf, each **value** is the book. You read a blend of all books, weighted by how well each label matches your question.
- *Heads are committee members.* Each head looks for a different kind of relation; the output matrix W^O is the chair who merges their reports.
- *Block = communicate, then think.* Attention moves information *between* tokens; the FFN processes each token *alone*.

Recall quiz (answers in Section 10).

1. What are the shapes of $QK^\top$ and of the attention output in (1)?
2. Why is there a $\sqrt{d_k}$ in the denominator?
3. What does the Transformer block do to the *order* of the tokens if no positional information is added?
4. What does a causal mask forbid?
5. Which of BERT, GPT, T5 uses cross-attention?

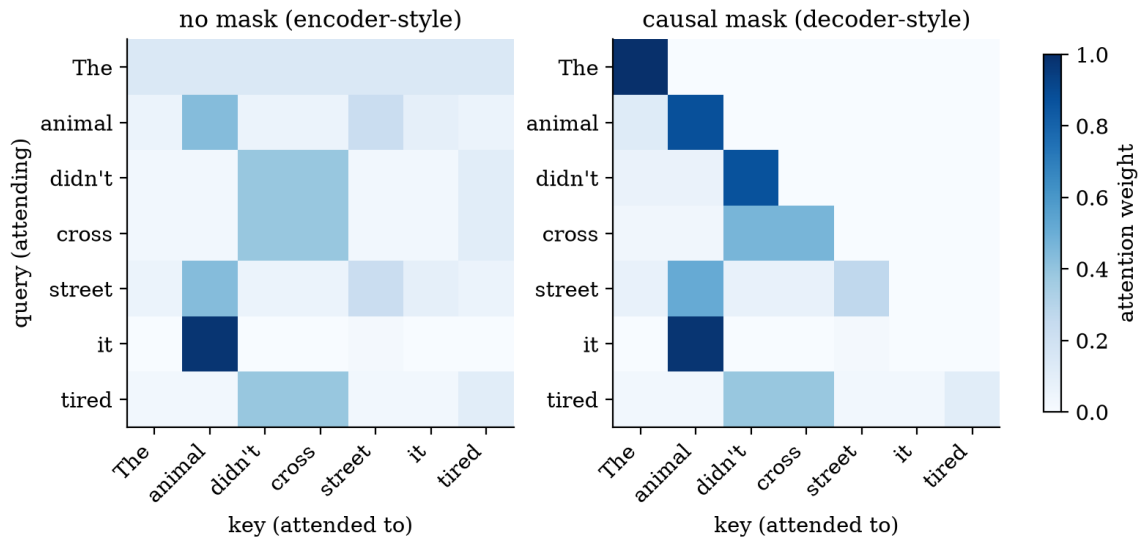
3. Understand: the derivation

3.1 Tier 1 (easy): intuition and a picture

A language task is a **sequence** problem: the meaning of a word depends on other words, possibly far away. In “The animal didn’t cross the street because *it* was tired”, what does *it* refer to? To answer, the representation of “it” has to *look back* at “animal”.

Before 2017 the standard tool was the **recurrent network**: read the words one at a time, keep a running memory (the hidden state), update it at each word. Two problems follow. (1) It is **sequential**: step t cannot start before step $t - 1$ finishes, so it cannot use a GPU’s parallelism over the sequence. (2) Information from the start has to survive many updates to reach the end.

Attention removes both problems with one idea: let every position look *directly* at every other position and take a weighted average of what it finds. The weights are computed from similarity. Figure 1 shows the result for a toy sentence.



Attention weights for a hand-built seven-token example (the vectors are set by hand to make the point, not learned). Rows are the token that is reading, columns the token being read. In both panels the row for “it” puts 0.98 of its weight on “animal”. On the right a causal mask (grey zeros above the diagonal) forbids looking at later words.

How to read it: each **row** sums to 1. Row “it” is almost one dark cell: “it” copies information from “animal”. Row “The” on the left is a flat stripe: its query is zero, so every score is equal and it averages everything (each weight $1/7 = 0.143$). On the right, row “The” is a single dark cell, because the mask leaves it only itself to read.

3.2 Tier 2 (medium): the definitions made precise

3.2.1 The recurrent network and its limits

An RNN keeps a hidden state $h_t \in \mathbb{R}^d$ and updates it with the new input x_t :

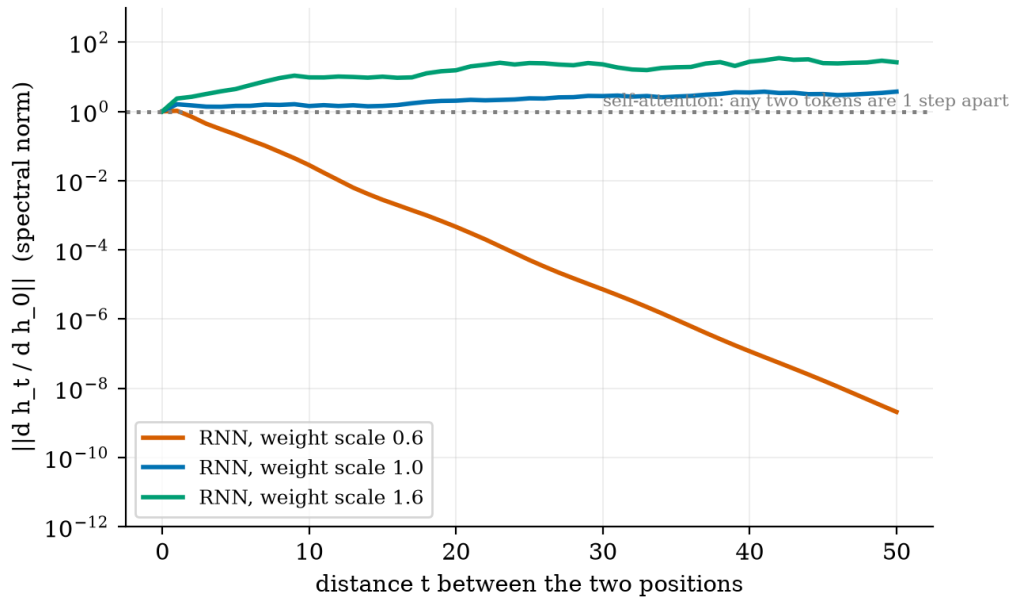
$$h_t = \tanh(Wh_{t-1} + Ux_t). \quad (4)$$

An LSTM (Hochreiter and Schmidhuber, 1997) adds gates that decide what to keep and what to overwrite, which helps, but the loop structure remains: h_t depends on h_{t-1} .

To learn a dependency between positions 0 and t , the gradient must flow back through t steps. By the chain rule, with Jacobians $J_s = \partial h_{s+1} / \partial h_s = \text{diag}(1 - h_{s+1}^2) W$,

$$\frac{\partial h_t}{\partial h_0} = J_{t-1} J_{t-2} \cdots J_0. \quad (5)$$

A product of t matrices. If each stretches a typical vector by a factor $\lambda < 1$ the product shrinks like λ^t (**vanishing gradient**); if $\lambda > 1$ it grows like λ^t (exploding). Figure 2 measures it.



Spectral norm of the Jacobian product in (5) for a tanh RNN with $d = 64$, as a function of the distance t (log scale). Weight scale 0.6: the norm is 0.028 at $t = 10$ and 2×10^{-9} at $t = 50$. Scale 1.0: 1.43 and 3.73. Scale 1.6: 9.6 and 26.3. Self-attention connects any two positions in one step.

The honest reading of Figure 2 is *not* “gradients always vanish”. For small weights they vanish by nine orders of magnitude in 50 steps; for larger weights the norm does not vanish at all, and at scale 1.6 it grows (the tanh saturates, which limits the growth). The point is that **the outcome depends on the weight scale in a way the training procedure does not control**, and the plotted number is the *largest* singular value, which says nothing about whether *useful* directions survive. Attention avoids the question: the path between two positions has length 1.

3.2.2 Attention, computed by hand

Take one query $Q = (1, 0)$, two keys $K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ and values $V = \begin{pmatrix} 10 & 0 \\ 0 & 10 \end{pmatrix}$, so $d_k = 2$.

$$\begin{aligned} \text{scores} &= QK^\top / \sqrt{2} = (1, 0) / 1.4142 = (0.7071, 0), \\ \text{weights} &= \text{softmax} = \left(\frac{e^{0.7071}}{e^{0.7071} + 1}, \frac{1}{e^{0.7071} + 1} \right) = \left(\frac{2.028}{3.028}, \frac{1}{3.028} \right) = (0.670, 0.330), \\ \text{output} &= 0.670(10, 0) + 0.330(0, 10) = (6.698, 3.302). \end{aligned}$$

The query matches the first key, so the output is mostly the first value, but not entirely: attention is a *soft* lookup. The module's `attention(Q, K, V)` returns exactly these numbers.

3.2.3 Self-attention, cross-attention, masks

In **self-attention** Q , K and V are all linear projections of the *same* sequence x : $Q = xW^Q$, $K = xW^K$, $V = xW^V$. In **cross-attention** the queries come from one sequence (for example the text being generated) and the keys and values from another (for example the encoded source sentence). The formula (1) is identical.

A **causal mask** sets the score of every forbidden pair to $-\infty$ *before* the softmax, so its weight becomes exactly 0. For a decoder that generates left to right, position i may read only positions $j \leq i$; otherwise training would let the model cheat by reading the very token it must predict.

3.3 Tier 3 (medium-hard): the main derivations

3.3.1 Why divide by $\sqrt{d_k}$

Let the components of a query q and a key k be independent with mean 0 and variance 1. The raw score is $s = q^\top k = \sum_{j=1}^{d_k} q_j k_j$.

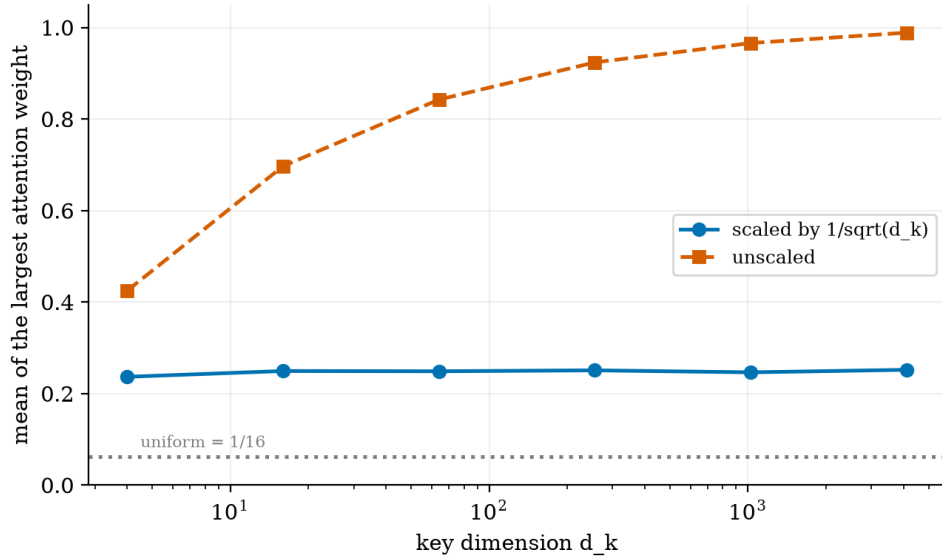
$$\begin{aligned} \mathbb{E}[s] &= \sum_j \mathbb{E}[q_j] \mathbb{E}[k_j] = 0 && \text{(independence)} \\ \text{Var}(s) &= \sum_j \text{Var}(q_j k_j) = \sum_j \mathbb{E}[q_j^2] \mathbb{E}[k_j^2] = d_k && \text{(terms are independent, each has variance 1)} \end{aligned}$$

So the typical size of a score grows like $\sqrt{d_k}$. Large scores push the softmax into saturation: one weight near 1, the rest near 0, and the gradient through the softmax is then nearly zero. Dividing by $\sqrt{d_k}$ restores $\text{Var}(s/\sqrt{d_k}) = 1$ for every d_k .

The test `test_sqrt_dk_keeps_score_variance_one` checks the same fact numerically with $d_k = 256$.

3.3.2 Multi-head attention costs no more than one big head

With h heads of width $d_k = d/h$, each head's projections are $d \times d_k$, so all $3h$ of them together hold $3h d d_k = 3d^2$ weights, exactly what one full-width head would hold. The output matrix adds d^2 . So **attention has $4d^2$ weights regardless of h** . What changes with h is the *structure*: h independent softmaxes, which lets different heads attend to different things at the same position.



Mean of the largest attention weight over 16 random keys, as d_k grows. With the $1/\sqrt{d_k}$ scaling (blue) it stays at 0.25 for every d_k . Without it (red) it climbs from 0.43 at $d_k = 4$ to 0.99 at $d_k = 4096$: the softmax becomes a hard argmax.

3.3.3 Positional encoding, and why the sinusoid gives relative offsets

Self-attention treats its input as a *set*: shuffle the tokens and the outputs are shuffled the same way (the model is permutation-equivariant), so “dog bites man” and “man bites dog” would look identical. Order must be injected. The original solution adds $PE_{pos} \in \mathbb{R}^d$ ((2)) to the embedding of the token at position pos .

Write $\omega_i = 10000^{-2i/d}$. The pair of entries for frequency i is $(\sin \omega_i pos, \cos \omega_i pos)$. By the angle-addition formulas,

$$\begin{pmatrix} \sin \omega_i (pos + k) \\ \cos \omega_i (pos + k) \end{pmatrix} = \begin{pmatrix} \cos \omega_i k & \sin \omega_i k \\ -\sin \omega_i k & \cos \omega_i k \end{pmatrix} \begin{pmatrix} \sin \omega_i pos \\ \cos \omega_i pos \end{pmatrix}. \quad (6)$$

So PE_{pos+k} equals a fixed **rotation** of PE_{pos} that depends on the offset k and *not* on pos . That is the property Vaswani et al. state in Section 3.5: a linear map takes PE_{pos} to PE_{pos+k} . It also gives the dot product between two positions a clean form:

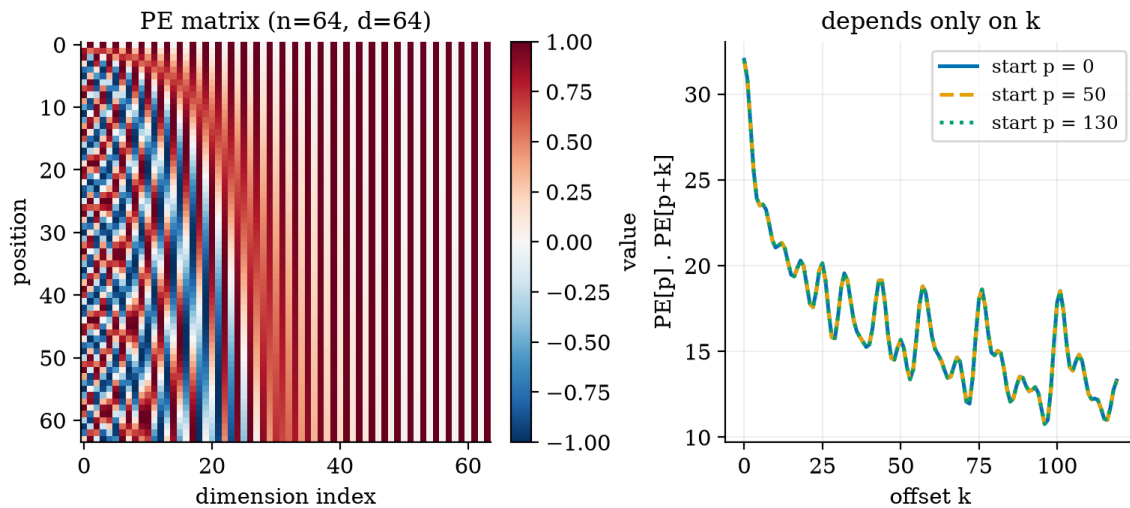
$$PE_{pos} \cdot PE_{pos+k} = \sum_{i=0}^{d/2-1} [\sin \omega_i pos \sin \omega_i (pos + k) + \cos \omega_i pos \cos \omega_i (pos + k)] = \sum_{i=0}^{d/2-1} \cos(\omega_i k). \quad (7)$$

It depends only on k . At $k = 0$ it equals $d/2$ (each term is 1): for $d = 64$ that is 32, and the code returns exactly 32.0. At $k = 10$ it is 21.05, from every starting position (the spread over 100 starting positions is 2×10^{-14}).

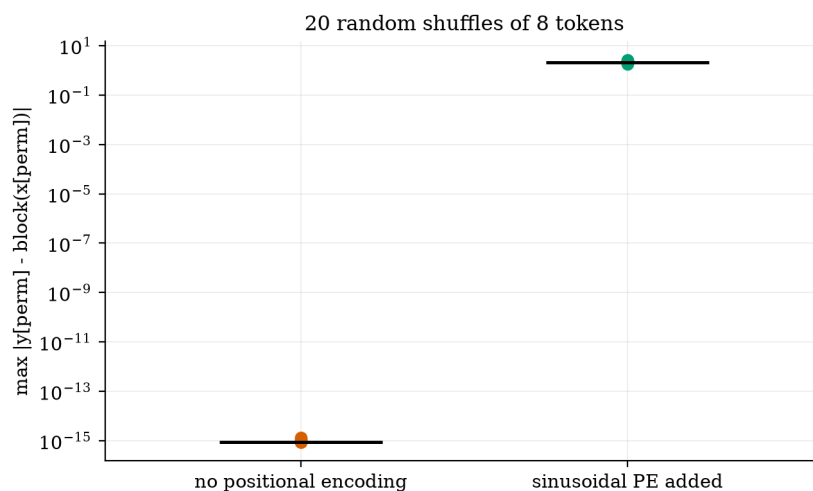
How much does order matter to a block? Figure 5 shuffles 8 tokens 20 times and measures how far the output moves from the correspondingly shuffled output of the unshuffled input.

3.3.4 Residuals, layer norm, and the parameter count of a block

The **residual connection** $x + f(x)$ gives gradients a direct path through every layer, so a stack of dozens of blocks stays trainable. **Layer normalisation** rescales each token vector to zero mean and



Left: the positional-encoding matrix for 64 positions and 64 dimensions; low dimensions oscillate fast, high dimensions slowly. Right: the dot product between position p and $p + k$ for three different starting positions p ; the three curves lie exactly on top of each other, as (7) predicts.



Largest difference after shuffling the token order (20 random shuffles, log scale). Without positional encoding the difference is 9×10^{-16} (rounding error only: the block cannot see order). With positional encoding added it is between 1.8 and 2.6.

unit variance, then applies a learned scale g and shift b : $\text{LN}(x) = g \odot (x - \mu) / \sqrt{\sigma^2 + \varepsilon} + b$.

Count the parameters of one block with width d and $d_{ff} = 4d$:

parameters of one block, $12d^2 + 13d$. The module's `count_params` on a block with $d = 16$ returns $3280 = 12 \cdot 256 + 13 \cdot 16$, and the test `test_param_count_formula` checks $d = 16$ and $d = 64$. This is the $12Ld^2$ used in Unit 3 for a model of L blocks.

part	weights	biases
attention W^Q, W^K, W^V, W^O	$4d^2$	$4d$
FFN W_1 ($d \times 4d$), W_2 ($4d \times d$)	$8d^2$	$4d + d$
two layer norms (g and b each)		$4d$
total	$12d^2$	$13d$

3.4 Tier 4 (hard): cost, and the three Transformer families

3.4.1 Where the FLOPs go

Count one multiply-add as 2 FLOPs, for a sequence of n tokens.

- **Dense part.** Q, K, V and output projections: four matmuls of $n \times d$ by $d \times d$, so $4 \cdot 2nd^2 = 8nd^2$. FFN: $2 \cdot 2ndd_{ff} = 16nd^2$ for $d_{ff} = 4d$. Total $24nd^2$, which is *linear* in n .
- **Attention scores.** $QK^\top$ is $n \times d$ (over all heads) times $d \times n$: $2n^2d$; multiplying the weights by V costs another $2n^2d$. Total $4n^2d$, which is *quadratic* in n .

The two are equal when $4n^2d = 24nd^2$, that is at

$$n^* = 6d. \quad (8)$$

For $d = 768$ that is $n^* = 4608$ tokens; the test `test_flops_crossover_is_six_d` checks the algebra.

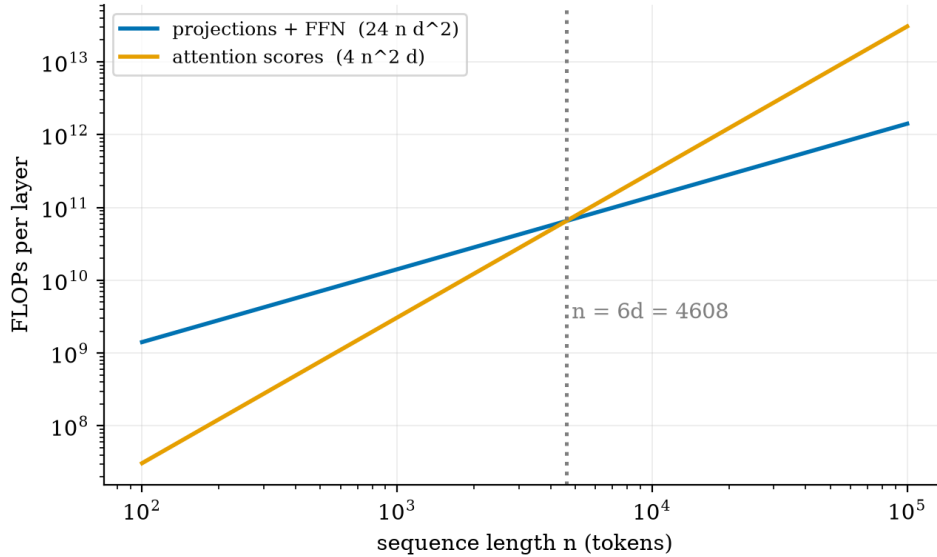
The numbers behind the figure: at $n = 1024$, attention scores 3.2 GFLOP versus dense 14.5 GFLOP; at $n = 4608$ both are 65.2 GFLOP; at $n = 32768$, 3299 versus 464 GFLOP. **Memory** is also quadratic: the weight matrix of one head at $n = 32768$ in 16-bit numbers is $32768^2 \times 2$ bytes = 2.1 GB, which is why long contexts are expensive (Unit 10).

Table 1 of Vaswani et al. makes the comparison with other layer types (their notation: n length, d width):

two rows of the paper's Table 1. Attention is *more* expensive per layer when $n > d$ but needs $O(1)$ sequential steps (parallel over positions) and connects any two positions in one step.

layer type	complexity per layer	sequential operations	maximum path length
self-attention	$O(n^2 d)$	$O(1)$	$O(1)$
recurrent	$O(nd^2)$	$O(n)$	$O(n)$

3.4.2 Encoder, decoder, encoder-decoder



FLOPs per layer against sequence length for $d = 768$ (log-log). The dense part (blue) is a straight line of slope 1, the attention scores (orange) a line of slope 2. They cross at $n = 6d = 4608$. At $n = 1024$ the scores are 18.2% of the layer; at $n = 32768$ they are 87.7%.

BERT (Devlin et al. 2018) is an encoder trained to fill in masked tokens; T5 (Raffel et al. 2019) casts every task as text in, text out, with an encoder and a decoder. The decoder-only form is the design behind most current chat models, which is why Unit 3’s next-token view applies to them directly.

family	attention pattern	trained by	examples	good at
encoder only	full (every token sees every token)	masked-token prediction	BERT	classification, search embeddings, tagging
decoder only	causal mask	next-token prediction	GPT-style models	open-ended generation, chat
encoder-decoder	encoder full; decoder causal plus cross-attention	text-to-text	original Transformer, T5	translation, summarisation

The **parameter sizes of the original model**. Table 3 of Vaswani et al. gives the base model $N = 6$ layers, $d = 512$, $d_{ff} = 2048$, $h = 8$, $d_k = d_v = 64$, with 65 million parameters. From the block count above, an encoder layer has $4d^2 + 2dd_{ff} = 3,145,728$ weights, a decoder layer one more attention ($+4d^2$) for 4,194,304; six of each is 44,040,192. Adding one shared embedding table of about $37,000 \times 512 = 18,944,000$ (the paper’s shared vocabulary is “about 37000” tokens) gives 62,984,192 ≈ 63 million, within 3% of the paper’s 65 million. I did not resolve the remaining 2 million (the exact vocabulary size in the authors’ code, or components I have not counted).

3.5 Tier 5 (hardest): open questions

1. **Is the sinusoidal encoding still the right idea?** The paper found learned positional embeddings gave “nearly identical results” (Section 3.5), and chose sinusoids in the hope that they would extrapolate to longer sequences than any seen in training. Many current models instead rotate the queries and keys by position-dependent angles (rotary embeddings, Su et al. 2021), which keeps the property of (7), that the score depends on the *offset*, but builds it into the attention score rather than adding it to the input. Which choice generalises best to very long contexts is still an active empirical question; I have not tested it here.
2. **Quadratic cost.** Figure 6 shows the scores dominate beyond $n = 6d$. Whether exact attention can be replaced by something cheaper without losing quality at scale is open; it matters for every long-context model.
3. **What do heads actually do?** In a random block the heads differ only by their random weights. In a trained model it is an empirical finding, not a theorem, that some heads track simple patterns (the previous token, the same word earlier in the text). Interpretability research tests such claims head by head.

4. Apply

4.1 Worked problems

Problem 1 (attention by hand). Reproduce Section 3.2.2 with $Q = (0, 1)$ instead. By symmetry the weights are (0.330, 0.670) and the output (3.302, 6.698).

Problem 2 (size of a model). GPT-2 small has $L = 12$ blocks of width $d = 768$. The block weights are $12 \cdot 12 \cdot 768^2 = 84,934,656$, as in Unit 3. The crossover length for these blocks is $n^* = 6 \cdot 768 = 4608$, so a context of 1024 tokens is still in the regime where the dense part (82% of the layer’s FLOPs, Figure 6) dominates.

Problem 3 (relative position). With $d = 64$, what is $PE_0 \cdot PE_{10}$? From (7) it is $\sum_{i=0}^{31} \cos(10 \cdot 10000^{-2i/64})$, and the code gives 21.0516 (the value is the same for $PE_{50} \cdot PE_{60}$: 21.0516).

Problem 4 (memory of the score matrix). One head, $n = 1024$, 16-bit values: $1024^2 \times 2 \approx 2.1$ MB. With 12 heads, 25.2 MB per layer per sequence, and the number grows 4 times each time the context doubles.

4.2 Code and figures

Everything is in `codes/llm/transformer_np/` (see its README). The core is attention plus one block:

```
def attention(Q, K, V, mask=None):
    scores = Q @ K.T / np.sqrt(Q.shape[-1])
    if mask is not None:
        scores = np.where(mask, scores, -1e30)    # forbidden pairs -> weight 0
    w = softmax(scores, axis=-1)
    return w @ V, w

def block(x, P, mask=None, memory=None):        # pre-LN: x + MHA(LN x), then x + FFN(LN x)
    a, w = mha(layer_norm(x, **P["ln1"]), layer_norm(x, **P["ln1"]), P["att"], P["h"], mask)
    x = x + a
```

...

All six figure scripts print the numbers quoted in this document.

Try it in the playground

The **Attention heatmap** widget lets you edit the sentence and pick which word is looking, with and without the causal mask. The **Positional encoding** widget shows the sinusoids and the dot-product-versus-offset curve for the width and number of positions you choose.

5. Analyze

Checklist items used: limits (5.1), dominance map (5.2), sensitivity (5.3), invariants (5.4), failure modes (5.5).

5.1 Limits

- $n = 1$: the softmax has one entry, weight 1, so attention returns the value itself.
- All scores equal (for example $Q = 0$): uniform average, as in row “The” of Figure 1.
- $d_k \rightarrow \infty$ without scaling: the softmax becomes an argmax (Figure 3).
- $h = 1$ versus $h = d$: the weight count is unchanged ($4d^2$), but $h = d$ would give width-1 heads whose scores are a product of two scalars, too little to express useful similarity.

5.2 Dominance map

The dense term $24nd^2$ and the score term $4n^2d$ trade places at $n = 6d$ (Figure 6). Below it, making the model wider is what costs; above it, making the context longer is.

5.3 Sensitivity

The attention output is most sensitive to the scale of Q and K . Doubling both scales the scores by 4 and sharpens the softmax sharply; this is the same mechanism as lowering the sampling temperature in Unit 3, now inside the model.

5.4 Invariants

- *Permutation equivariance without PE.* Verified to 9×10^{-16} (Figure 5, and the permutation test of the module).
- *No future leakage with a causal mask.* Changing tokens 5 and later does not change outputs 0 to 4; the test `test_causal_no_future_leak` checks it.
- *Row sums.* Every attention row sums to 1 (`test_attention_rows_sum_to_one_and_mask`).

5.5 Failure modes

- **Long-context dilution.** With n keys a head must spread weight over many positions; a relevant token can be drowned by many weakly matching ones.

- **Quadratic memory** for the score matrix (Problem 4).
- **Position extrapolation.** A model trained on lengths up to $n_{\max}$ may behave badly beyond it, whichever encoding is used.
- **Masking mistakes.** Forgetting the causal mask in a decoder leaks the answer during training, and the loss falls suspiciously fast.

6. Evaluate

Transformer versus recurrent network. The Transformer wins on parallelism (all positions at once on a GPU), path length (1 versus n), and in practice on quality at scale. The recurrent network wins on memory at inference: it stores a fixed-size state, while a Transformer must keep all past keys and values (Unit 10). Both facts are in Table 1 of the paper in different words.

Inductive bias. A convolution assumes locality; an RNN assumes order and a running summary; self-attention assumes almost nothing except that content-based lookup is useful. Less built-in structure means more data is needed, and more flexibility once it is available. That trade-off is the quiet reason scale (Unit 3) mattered so much for Transformers.

When not to use full attention. Very long sequences (hundreds of thousands of tokens) with a tight memory budget; small data sets where a model with a stronger bias, such as a convolution for images of a few hundred samples, will do better.

Wider context. Attention is a differentiable form of *associative memory*: a soft dictionary lookup. That framing connects the Transformer to older ideas about content-addressable memory, and to retrieval in Unit 5, where the “keys” are documents.

7. Create

What you can build from this

1. **Attention visualiser:** feed any sentence through `block` with random weights and plot the heads; then replace the weights with hand-built ones, as in Figure 1, to make a given word attend to another.
2. **Order test:** show that bag-of-words ($PE = 0$) cannot separate “dog bites man” from “man bites dog” and that adding `sinusoidal_pe` can.
3. **Cost calculator:** a function that returns the crossover length and the FLOP split for any (d, d_{ff}, n) , and plot it for $d = 512, 768, 4096$.
4. **Combine with Unit 3:** put `block` under the `sample` function from Unit 3 to build a (random-weight) text generator with the right *interfaces*; training it is the next step.
5. **Module for the open-source package:** add rotary embeddings to `transformer_np` with a test that the score depends only on the offset.

8. Visualization lesson

charts taught here; log them in the visualization log. How to read the log-log plot: a line of slope 1 means cost $\propto n$, slope 2 means $\propto n^2$; where two lines cross, the dominance changes.

Chart type	Best for	How it misleads	matplotlib call
heatmap (<code>imshow</code>)	a matrix of weights (Fig. 1, 4 left)	colour scale hides small values; a sequential map implies order	<code>ax.imshow(M, cmap="Blues", vmin=0, vmax=1)</code>
diverging heatmap	values with a meaningful zero (PE matrix)	the midpoint must be 0 or the colours lie	<code>cmap="RdBu_r", vmin=-1, vmax=1</code>
line on semilog axis	quantities spanning many orders (Fig. 2)	straight lines look like exponentials only if the data are	<code>ax.semilogy(x, y)</code>
log-log line	power laws; slopes are exponents (Fig. 6)	only slopes are readable, not differences	<code>ax.loglog(x, y)</code>
strip plot with medians	a few repeats of an experiment (Fig. 5)	hides the distribution if points overlap	<code>ax.semilogy([i]*len(v), v, "o")</code>

9. Problem set

Graded exercises

- (easy) With $Q = (0, 1)$, $K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $V = \begin{pmatrix} 10 & 0 \\ 0 & 10 \end{pmatrix}$ compute the attention output.
- (medium) A model has $d = 1024$, $d_{ff} = 4096$. Compute the parameters of one block (weights only) and the sequence length n^* at which attention scores equal the dense FLOPs.
- (medium-hard) Prove $PE_{pos} \cdot PE_{pos+k} = \sum_i \cos(\omega_i k)$ from the definition, and show that the value at $k = 0$ is $d/2$.
- (hard) Show that attention without positional encoding is permutation-equivariant: if P is a permutation matrix, $\text{Attention}(PQ, PK, PV) = P \text{Attention}(Q, K, V)$. (Hint: the softmax acts on each row, and $PQK^\top P^\top$ is a re-indexing of $QK^\top$.)
- (hardest) For a decoder-only model with a causal mask, show that the attention-score FLOPs per layer over a full sequence are about *half* of $4n^2d$ if the masked entries are not computed. Why do many implementations still compute them?

10. Retrieval practice and recap

Answers to the Section 2 quiz.

- $QK^\top$ is $n_q \times n_k$; the output is $n_q \times d_v$.
- Raw dot-product scores have variance d_k ; dividing by $\sqrt{d_k}$ keeps the variance at 1 so the softmax does not saturate (Section 3.3.1).
- Nothing: the outputs are permuted exactly like the inputs, so the model cannot see order (Figure 5).

4. Attention from a position to any *later* position.
5. T5 (and the original Transformer): the encoder-decoder models. BERT is encoder only; GPT-style models are decoder only and have no cross-attention.

Explain it to a friend.

- Why can a Transformer connect “it” to “animal” in one step when an RNN needs many?
- What would break if you removed the positional encoding?
- Why does making the context 4 times longer make attention 16 times more expensive in the score computation?

Re-derive tomorrow without looking: (a) $\text{Var}(q^\top k) = d_k$; (b) $PE_{pos} \cdot PE_{pos+k} = \sum_i \cos \omega_i k$; (c) $12d^2$ parameters per block and the crossover $n^* = 6d$.

11. Where next

- **Next:** Unit 3 (large language models: tokens, embeddings, decoding and scaling), which uses the block you have just assembled as the network inside an LLM; Units 4 to 10 then build applications on top.
- **Terms met but not taught:** rotary embedding, FlashAttention, grouped-query attention, KV cache (Unit 10).
- **Open threads:** the 2 million parameters between the paper’s 65M and the 63M count in Section 3.4.2; which positional scheme extrapolates best (Section 3.5).